



Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias
Ingeniería en Ciencias de Datos y Matemáticas

Uso de Álgebras Modernas para Seguridad y
Criptografía Bloque MA2006B · Grupo 601 · Equipo 1

MANUAL DEL PROGRAMADOR **Sistema de Gestión de Identidades Digitales para Casa Monarca** *Manual de programador*

Alicia Josefina de la Garza Montelongo — A01198742

Gisel Regina Benítez Calvillo — A00228137

Lillianne Sepúlveda Cuevas — A00839109

Axel Xavier Olivar Lozano — A01286176

Jorge Eduardo Avila Montoya — A01410275

Profesores del bloque:

Raúl Gómez · Anas Wajid · Alberto F. Martínez

Socio Formador: Casa Monarca – Ayuda Humanitaria al Migrante, A.B.P.

Monterrey, Nuevo León, a 12 de junio de 2026

Introducción

Este documento está dirigido a desarrolladores, administradores de sistemas y futuros equipos de trabajo que necesiten instalar, configurar, extender o mantener el Sistema de Gestión de Identidades Digitales desarrollado para Casa Monarca – Ayuda Humanitaria al Migrante, A.B.P.

El sistema implementa una Infraestructura de Clave Pública (PKI) simplificada que permite gestionar identidades digitales, emitir certificados, firmar documentos y verificar autenticidad mediante criptografía asimétrica RSA.

Objetivos

- Facilitar la instalación y configuración del entorno de desarrollo.
- Servir como referencia de arquitectura y estructura interna del proyecto.
- Documentar las dependencias, módulos y flujos técnicos relevantes.
- Estandarizar el proceso de despliegue en entornos locales y en la nube.
- Permitir futuras extensiones y mantenimiento por equipos distintos al original.

Arquitectura General del Sistema

La solución sigue una arquitectura cliente-servidor en dos capas:

Servidor (backend): API REST desarrollada en Flask, que gestiona la lógica de negocio, la criptografía, la autenticación y la persistencia en SQLite. Puede ejecutarse de forma local o desplegarse en Railway.

Cliente (frontend): Aplicación de escritorio desarrollada en Tkinter que consume los servicios del servidor mediante HTTP. Se distribuye como ejecutable de Windows (.exe) generado con PyInstaller.

La comunicación entre cliente y servidor se realiza a través de HTTP con autenticación mediante tokens Bearer. Al iniciar el cliente, se solicita la URL del servidor (por ejemplo, `http://localhost:5000` o `http://192.168.1.5:5000`), la cual queda guardada para sesiones futuras.

Requisitos de Hardware

Máquina cliente:

- Procesador Intel Core i5 o superior.
- 8 GB de memoria RAM.
- 500 MB de espacio libre en disco.
- Acceso a la red donde corre el servidor.

Máquina servidor:

- Procesador de 2 núcleos o superior.
- 4 GB de RAM mínimo.
- 2 GB de almacenamiento disponible.
- Acceso permanente a Internet (si se despliega en Railway).

Requisitos de Software

- Python 3.10 o superior.
- SQLite 3 (incluido en Python estándar).
- Git.
- Visual Studio Code (recomendado como editor).
- Railway CLI o acceso al dashboard de Railway (para despliegue en nube).
- Sistema operativo: Windows 10/11 o Linux Ubuntu 20.04+.

Dependencias del Proyecto

Las siguientes bibliotecas deben instalarse en el entorno virtual:

Bibliotecas externas:

Flask, gunicorn, cryptography, requests, python-docx, lxml, pypdf, reportlab, Pillow, tzdata, PyInstaller.

Bibliotecas estándar de Python (no requieren instalación):

hashlib, sqlite3, secrets, json, base64, urllib.

Todas las dependencias externas están declaradas en requirements.txt.

Procedimiento de Instalación y Configuración

1. Clonar el repositorio

Abre una terminal y ejecuta:

```
bashgit clone https://github.com/<usuario>/<repositorio>.git
```

```
cd <nombre-del-repositorio>
```

2. Crear el entorno virtual

Dentro del directorio raíz del proyecto, crea un entorno virtual para aislar las dependencias:

```
bash# Windows
```

```
python -m venv venv
```

Linux / macOS

```
python3 -m venv venv
```

3. Activar el entorno virtual

```
bash# Windows (PowerShell)
```

```
venv\Scripts\Activate.ps1
```

Windows (CMD)

```
venv\Scripts\activate.bat
```

Linux / macOS

```
source venv/bin/activate
```

Al activarse correctamente, el prompt de la terminal mostrará el prefijo (venv).

4. Instalar dependencias

```
bashpip install -r requirements.txt
```

Este comando descarga e instala todas las bibliotecas necesarias declaradas en requirements.txt.

5. Configurar variables de entorno

Crea un archivo .env en la raíz del proyecto con las siguientes variables (o configúralas directamente en tu entorno):

envPORT=5000

RESET_SECRET=<clave-secreta-para-reset>

RAILWAY_VOLUME_MOUNT_PATH=<ruta-de-volumen-si-se-usa-Railway>

PORT: puerto en el que correrá el servidor Flask. Por defecto 5000.

RESET_SECRET: clave utilizada para proteger el endpoint de reset de base de datos.

RAILWAY_VOLUME_MOUNT_PATH: ruta donde Railway monta el volumen persistente (solo necesaria en despliegue en la nube).

6. Ejecutar el servidor

bash# Desarrollo local

python server/app.py

O usando Flask directamente

flask --app server/app.py run --port 5000

El servidor quedará escuchando en http://localhost:5000. Puedes verificarlo accediendo a esa URL desde el navegador; deberías recibir una respuesta JSON de estado.

7. Ejecutar el cliente

bashpython ui/main.py

Al iniciarse, el cliente pedirá la URL del servidor. Ingresa http://localhost:5000 si el servidor corre en la misma máquina. La URL quedará guardada para la próxima sesión.

Estructura del Proyecto y Descripción de Carpetas

raíz/

|

|— auth/

|— crypto/

|— db/

|— server/

|— client/

|— ui/

|— config/

|

|— requirements.txt

|— SistemalDentidad.spec

|— build.bat

|— .env

auth/ — Autenticación y Control de Acceso

Contiene la lógica de autenticación de usuarios y la gestión de permisos por rol. Aquí se validan los tokens Bearer que acompañan cada solicitud HTTP y se verifican los niveles de acceso (externo, operativo, coordinador, administrador).

Archivos clave: validación de tokens, decoradores de permisos, manejo de sesiones.

crypto/ — Criptografía

Núcleo criptográfico del sistema. Implementa la generación de pares de claves RSA-2048, la emisión de certificados digitales, la firma de documentos con RSA-PSS y la verificación de firmas. También incluye la lógica de la Lista de Revocación de Certificados (CRL).

Algoritmos utilizados:

RSA-2048 para criptografía asimétrica.

RSA-PSS para firmas digitales.

PBKDF2-HMAC-SHA256 para almacenamiento seguro de contraseñas.

SHA-256 para verificación de integridad.

db/ — Base de Datos y Persistencia

Capa de acceso a datos. Contiene todas las consultas SQL y el esquema de la base de datos SQLite. Gestiona las tablas de usuarios, certificados, claves públicas, solicitudes de firma, rutas de firmas, certificados revocados y registros de auditoría.

No se accede a la base de datos desde ningún otro módulo directamente; toda lectura y escritura pasa por las funciones definidas aquí.

server/ — API REST (Backend)

Define todos los endpoints HTTP que expone el sistema. Actúa como punto de entrada del servidor. Los controladores de ruta invocan la lógica de auth/, crypto/ y db/ para procesar cada solicitud.

Endpoints principales agrupados por funcionalidad:

Autenticación: login, logout, validación de token.

Usuarios: registro, aprobación, baja, revocación.

Certificados: emisión, descarga, consulta de CRL.

Claves: entrega segura de clave privada (única vez), gestión de clave pública.

Documentos: firma, verificación, descarga de documentos firmados.

Flujo de firmas: canalización, firma en cadena, consulta de estado.

Administración: logs, ruta de firmas, solicitudes pendientes.

client/ — Capa de Comunicación HTTP

Abstrae todas las llamadas HTTP que el cliente de escritorio hace al servidor. En lugar de que la interfaz llame directamente a requests, este módulo centraliza la construcción de URLs, el manejo de headers de autenticación, el procesamiento de respuestas y el manejo de errores de red.

Esto hace que cambiar la URL base del servidor o agregar nuevos endpoints no requiera modificar la interfaz de usuario.

ui/ — Interfaz Gráfica (Frontend)

Aplicación de escritorio construida en Tkinter. Contiene todas las ventanas, diálogos y paneles que ve el usuario. Se comunica exclusivamente con client/ para obtener o enviar datos; no contiene lógica de negocio ni criptografía.

Componentes principales:

Diálogo de conexión al servidor.

Pantalla de inicio de sesión y creación de cuenta.

Diálogo de descarga obligatoria de clave privada.

Panel principal con tarjetas por rol (externo, operativo, coordinador, administrador).

Módulos de solicitud, firma, verificación y seguimiento de documentos.

Panel de administración (alta, revocación, baja, ruta de firmas, logs).

config/ — Configuración General

Centraliza los parámetros de configuración del sistema: lectura de variables de entorno, constantes globales (nombre del sistema, versión, rutas de archivos internos) y configuración de seguridad (longitudes de claves, algoritmos permitidos, tiempo de vida de tokens).

Base de Datos

El sistema usa SQLite como motor de base de datos. El archivo se genera automáticamente en el primer arranque del servidor.

Tablas principales:

TablaDescripciónusuariosRegistro de todos los usuarios con su rol, estado y hash de contraseña.certificadosCertificados digitales emitidos por el sistema, vinculados a cada usuario.claves_publicasClaves públicas RSA asociadas a cada identidad.solicitudes_firmaSolicitudes de firma creadas por usuarios externos u operativos.rutas_firmaDefinición del orden de coordinadores en la firma en cadena.certificados_revocadosLista de Revocación de Certificados (CRL).logs_auditoriaRegistro cronológico de todas las acciones relevantes del sistema.

Procedimiento de Compilación del Cliente

El cliente puede empaquetarse como ejecutable de Windows (.exe) usando PyInstaller, eliminando la necesidad de que el usuario final tenga Python instalado.

Pasos para compilar:

1. Asegúrate de tener el entorno virtual activado y PyInstaller instalado:
2. `bash pip install pyinstaller`
3. Ejecuta el script de compilación:
 - a. `bash # Windows`
 - b. `build.bat`

O directamente con PyInstaller usando la especificación del proyecto:

```
bash pyinstaller SistemalDentidad.spec
```

El ejecutable resultante se generará en la carpeta `dist/`. El archivo `SistemalDentidad.spec` define los módulos incluidos, los archivos de datos adicionales (imágenes, configuración) y el nombre del ejecutable final.

Distribuye únicamente el contenido de `dist/` al usuario final.

Nota: la compilación debe realizarse en Windows para obtener un .exe compatible. Para Linux, el proceso produce un binario nativo equivalente.

Despliegue

Despliegue local:

Para entornos de desarrollo o uso en red interna, basta con ejecutar el servidor directamente con Python (ver sección de instalación). Cualquier cliente en la misma red puede conectarse usando la IP de la máquina servidora: `http://[IP-local]:5000`.

Despliegue en Railway:

Railway permite mantener el servidor accesible desde internet sin administrar infraestructura propia.

Crea un proyecto en `railway.app` y conecta el repositorio de GitHub.

Define las variables de entorno en el panel de Railway:

- PORT
- RESET_SECRET
- RAILWAY_VOLUME_MOUNT_PATH (añade un volumen persistente para la base de datos SQLite)

Railway detecta automáticamente el `requirements.txt` y despliega con `gunicorn`:

- `bash gunicorn server.app:app --bind 0.0.0.0:$PORT`

Railway asigna una URL pública (por ejemplo, `https://sistema-identidad.up.railway.app`). Usa esa URL en el cliente al conectarse.

Importante: configura un volumen persistente en Railway para que la base de datos SQLite no se pierda en cada redeploy. Sin volumen, los datos se borran cada vez que el servicio se reinicia.

Seguridad

Mecanismo de uso en el sistema: PBKDF2-HMAC-SHA256 Almacenamiento de contraseñas (hashing con sal). RSA-2048 Generación de par de claves por usuario; criptografía asimétrica. RSA-PSS Esquema de firma digital de documentos. SHA-256 Verificación de integridad de documentos firmados. Token Bearer Autenticación de sesiones entre cliente y servidor.

La clave privada de cada usuario se genera en el servidor, se entrega al usuario una única vez a través de un endpoint protegido y se elimina inmediatamente del servidor.

tras la descarga. El servidor conserva únicamente la clave pública. Esto implica que si el usuario pierde su archivo .pem, la clave no puede recuperarse y deberá crearse una nueva identidad.

Mantenimiento

Respallos de base de datos: realizar copias periódicas del archivo .sqlite (especialmente antes de actualizaciones). En Railway, los volúmenes persistentes permiten automatizar esto.

Logs de auditoría: revisar regularmente la tabla logs_auditoria o la pantalla de Logs en el panel de administrador para detectar actividad anómala.

Actualización de dependencias: ejecutar periódicamente `pip list --outdated` y actualizar con precaución, verificando compatibilidad especialmente en el módulo cryptography.

Revisión de seguridad: rotar el valor de RESET_SECRET periódicamente y revisar que los endpoints de administración permanezcan protegidos.

Solución de Problemas Comunes

Error: módulo no encontrado al ejecutar el servidor

Verifica que el entorno virtual esté activado ((venv) en el prompt) y que las dependencias estén instaladas: `pip install -r requirements.txt`.

El cliente no puede conectarse al servidor

Confirma que el servidor esté corriendo y escuchando en el puerto correcto. Verifica que no haya un firewall bloqueando el puerto 5000. Si usas Railway, asegúrate de que la URL pública sea correcta y que el servicio esté activo.

El usuario no puede iniciar sesión después de registrarse

Las cuentas nuevas quedan en estado "pendiente". Un administrador debe aprobarlas y asignarles un rol desde el panel de administración.

Error al firmar: no se puede cargar la clave privada

El archivo .pem seleccionado no corresponde al usuario, está corrupto o tiene permisos de lectura incorrectos. Verifica que sea el archivo original descargado al activar la cuenta.

Pérdida de clave privada

No existe mecanismo de recuperación. El administrador debe revocar la identidad actual del usuario y crear una nueva.

Error de permisos en Linux al ejecutar el servidor

Asegúrate de que el usuario tiene permisos de escritura en el directorio donde se crea la base de datos. Ejecuta `chmod 755 <directorio>` si es necesario.

Redeploy en Railway borra los datos

El sistema de archivos de Railway no es persistente por defecto. Debes añadir un volumen y apuntar `RAILWAY_VOLUME_MOUNT_PATH` a la ruta donde se guarda el archivo SQLite.

El ejecutable compilado no inicia en otra máquina Windows

Verifica que la compilación se haya hecho en la misma arquitectura (x64). Si hay librerías de sistema faltantes, redistribuye también los archivos de la carpeta `dist/` completa, no solo el `.exe`.

Recomendaciones para Desarrolladores

Separación de responsabilidades: no mezclar lógica de negocio, criptografía o acceso a datos dentro de `ui/`. La interfaz solo llama a `client/`, que llama al servidor.

Documentar nuevas funciones: cualquier endpoint nuevo debe documentarse con su método HTTP, ruta, parámetros esperados, respuesta exitosa y posibles errores.

Pruebas unitarias: implementar pruebas para los módulos `crypto/` y `db/` antes de hacer cambios en algoritmos o esquema de datos.

Variables de entorno para todo lo sensible: no hardcodear claves, rutas absolutas ni configuraciones de entorno directamente en el código.

Versionado del esquema de base de datos: si se añaden tablas o columnas, incluir scripts de migración para no romper instalaciones existentes.

No modificar el archivo `.pem` de un usuario desde el backend: el principio fundamental del sistema es que el servidor nunca retiene claves privadas. Cualquier funcionalidad nueva debe respetar este invariante.